

WebNet1

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

We found this packet capture and key. Recover the flag.

Hints

- Try using a tool like Wireshark.
 - How can you decrypt the TLS stream?
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - **Wireshark** – Packet analysis, TLS decryption, and HTTP object export
 - `grep` – Filter output
-

Complete Solution

Step 1: Download and Prepare

Download the files from the challenge page:

- `capture.pcap` – Network packet capture (encrypted TLS traffic)
- `picopico.key` – RSA private key for decrypting TLS

Step 2: Basic File Inspection

First, verify the file types:

```
file capture.pcap picopico.key
```

Expected output:

```
capture.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4
(Ethernet, capture length 65535)
picopico.key: OpenSSH private key (no password)
```

The `capture.pcap` is a standard **PCAP** file containing network traffic.

The `picopico.key` is an **OpenSSH private key** (RSA key, no password).

Step 3: Search for Visible Text

Check if the flag appears as plain text within any file:

```
strings capture.pcap | grep -i "picoCTF"
```

Output: (no results)

The traffic is encrypted – the flag isn't directly visible.

Step 4: Inspect Metadata

```
exiftool capture.pcap
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : capture.pcap
Directory                   : .
File Size                    : 93 kB
...
```

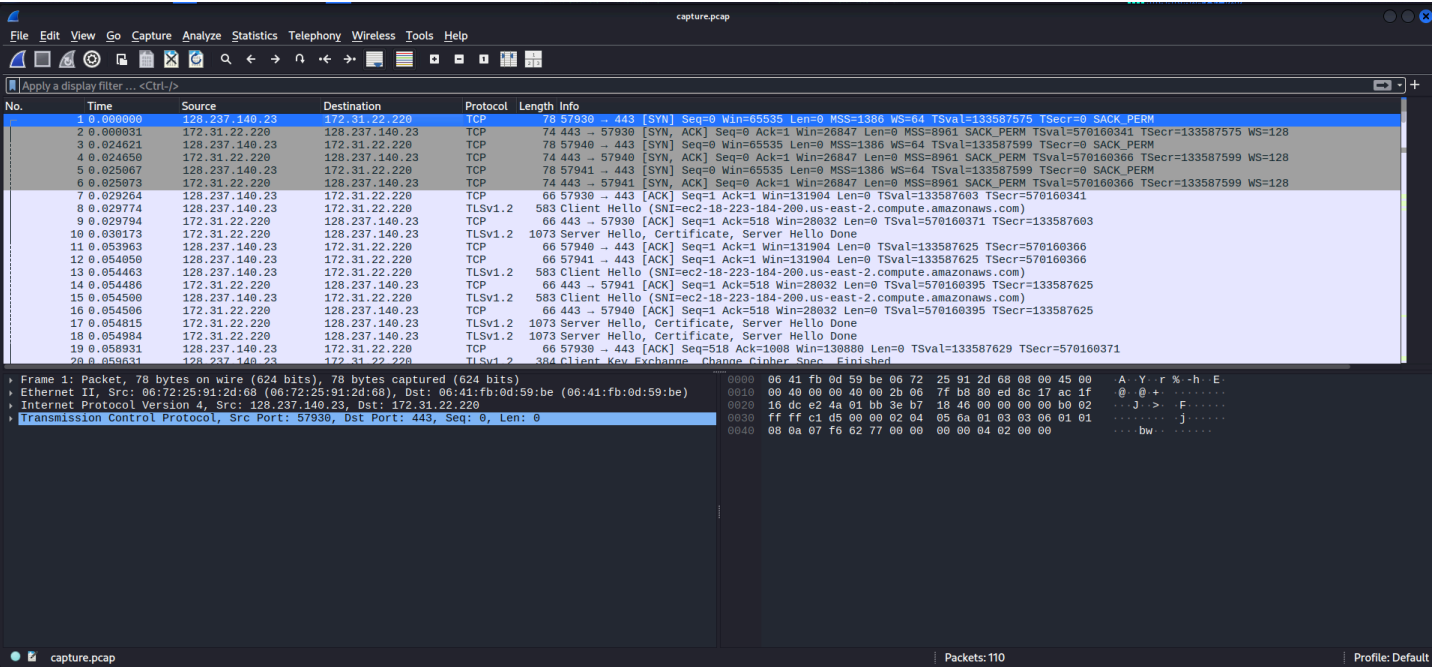
File Type : PCAP
Link Type : BSD Loopback
Time Stamp : 2019:08:23 18:27:03.908370+02:00

Nothing suspicious in the metadata.

Step 5: Open the PCAP in Wireshark

Launch Wireshark and open `capture.pcap` :

```
wireshark capture.pcap
```



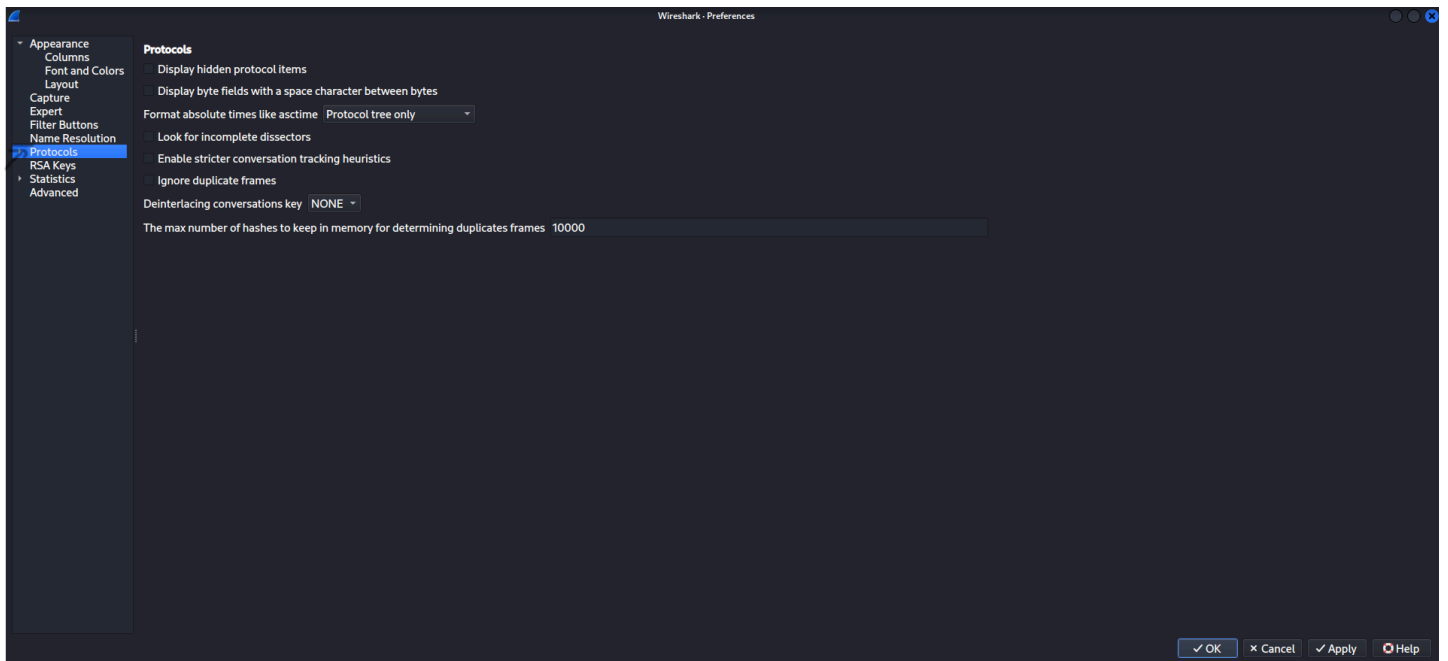
Initially, all traffic appears as “**Application Data**” – this is because the TLS (SSL) stream is encrypted. We need to decrypt it using the provided RSA private key.

Step 6: Configure TLS Decryption in Wireshark

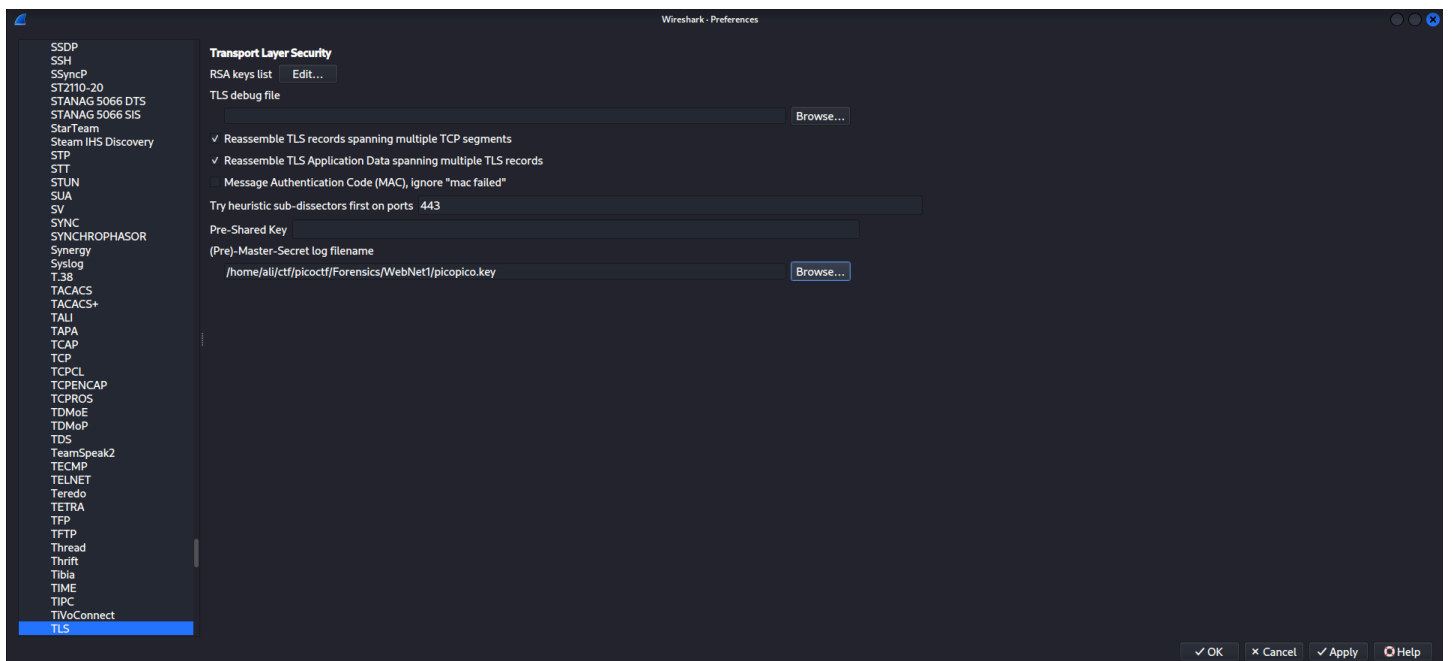
Wireshark can decrypt TLS traffic if you provide the server’s private key.

Step-by-step:

1. Go to **Edit** → **Preferences** (or press `Ctrl+Shift+P`)



2. Expand **Protocols** and scroll down to **TLS** (or **SSL** in older Wireshark versions)
3. In the “**(Pre)-Master-Secret log filename**” section, click the **Browse** button



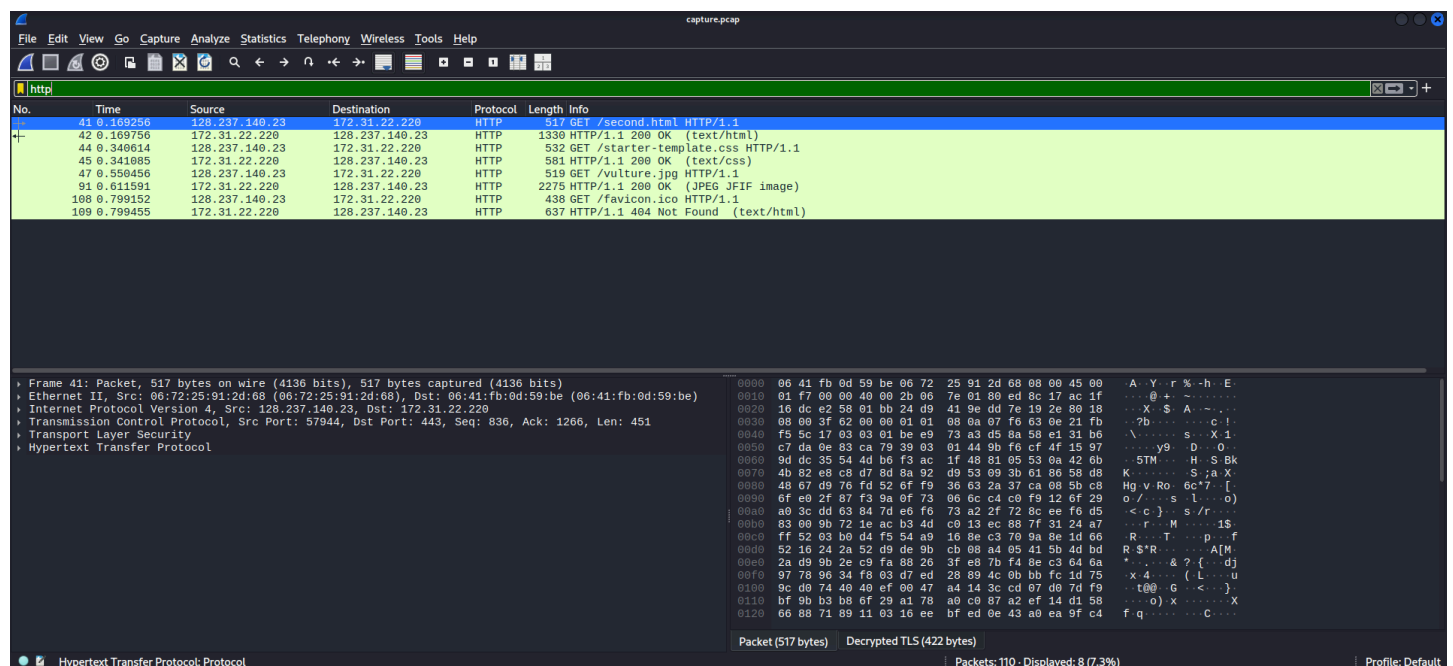
4. Add a new entry with the following:
 - **IP Address:** leave blank (applies to all IPs)
 - **Port:** 443 (default HTTPS port)
 - **Protocol:** http
 - **Key File:** browse and select `picopico.key` (or type the full path)
5. Click **OK** to save

Step 7: Verify Decryption

After configuring the key, Wireshark will automatically re-analyze the packets. The encrypted “Application Data” will now be displayed as **HTTP** traffic.

Filter for HTTP traffic:

http



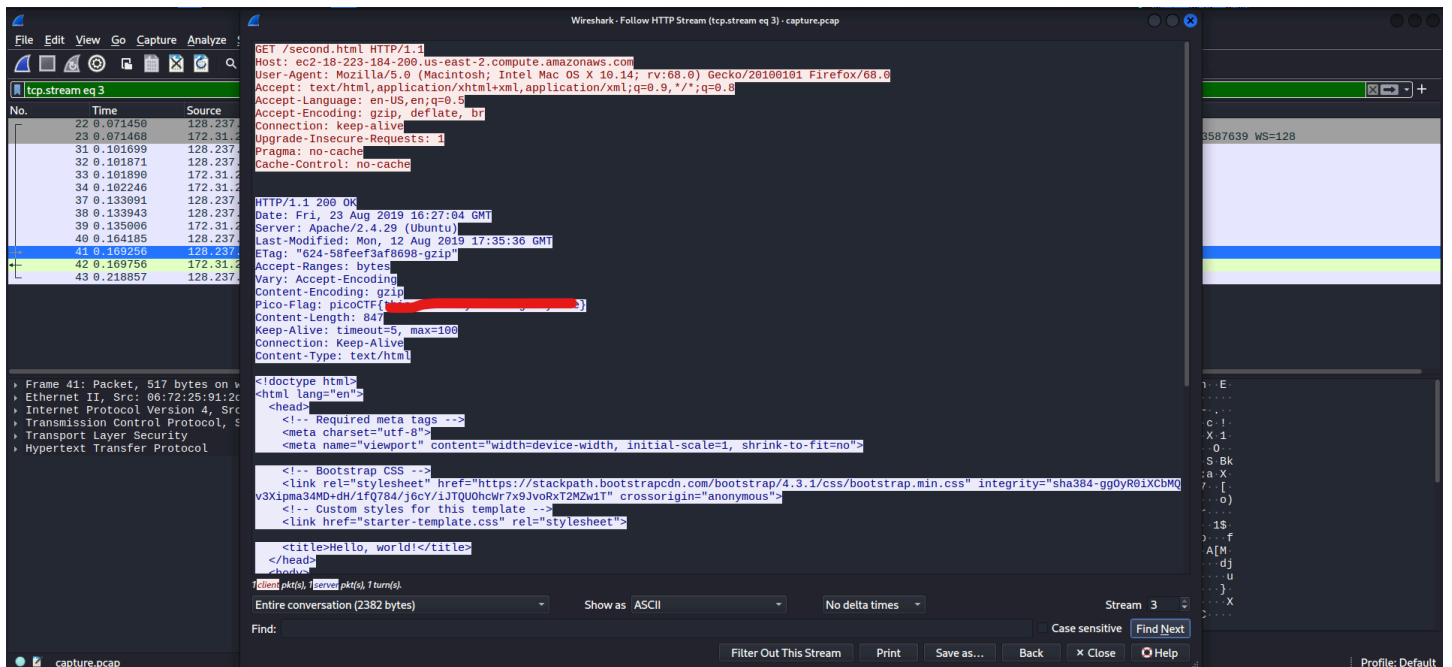
Step 8: Locate the Flag – First Attempt

Scroll through the HTTP responses or use the **Follow HTTP Stream** feature:

1. Right-click on any HTTP packet
2. Select **Follow** → **HTTP Stream**

Look for the **Pico-Flag** header in the response:

Pico-Flag: picoCTF{...}



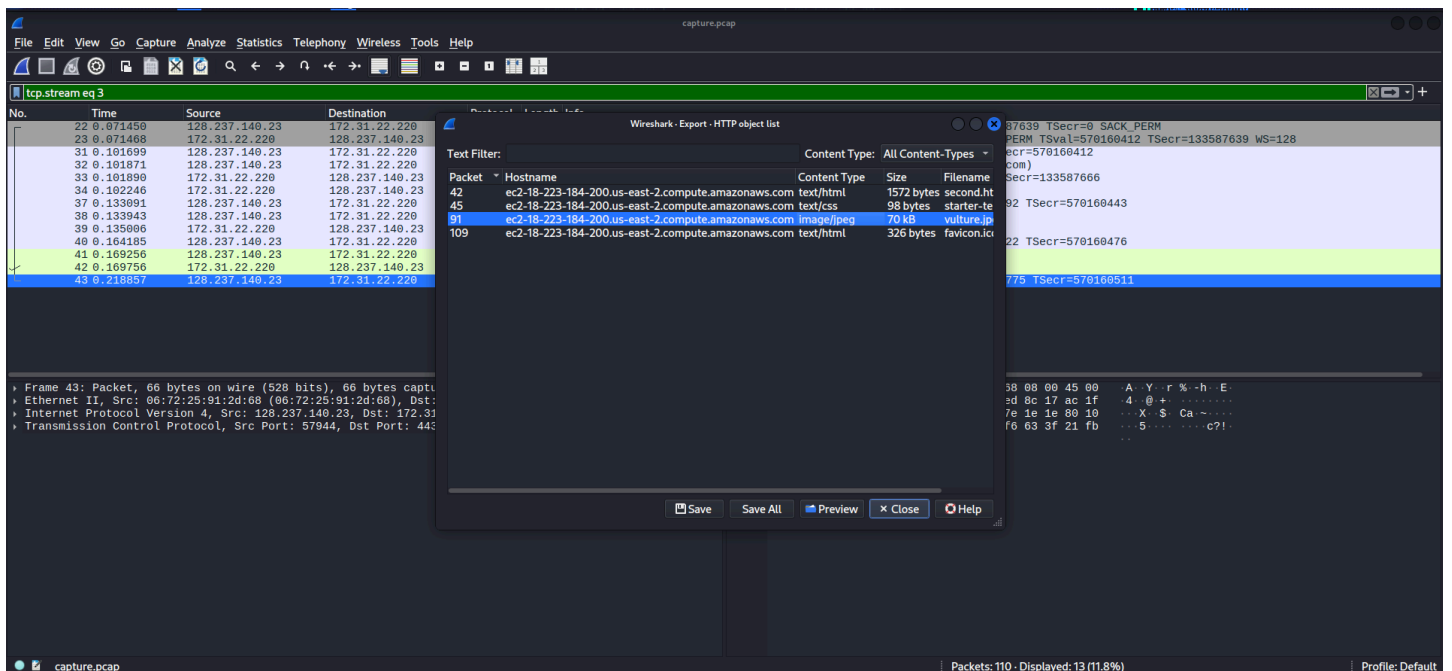
However, after submitting this flag, it was **incorrect**. There must be more to the challenge.

Step 9: Export HTTP Objects

Since the flag wasn't in the headers, let's look for transferred files.

In Wireshark:

1. Go to **File** → **Export Objects** → **HTTP**



2. A list of HTTP objects transferred during the session appears
3. Look for interesting files – in this case, an image named **vulture.jpg**



“sir, look at my pantaloons.”

“sir, you also look at my pantaloons.”

4. Select `vulture.jpg` and click **Save** (or **Preview** to view it directly)

Step 10: Analyze the Extracted Image

`file vulture.jpg`

Output:

vulture.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, Exif Standard: [TIFF image data, big-endian, direntries=5, xresolution=74, yresolution=82, resolutionunit=1], baseline, precision 8, 640x716, components 3

The image is a valid JPEG.

Step 11: Extract Strings from the Image

```
strings vulture.jpg | grep -i "picoCTF"
```

Output:

```
picoCTF{<REDACTED>}
```

Bingo! The flag is embedded as a string within the JPEG image.

Step 12: Alternative – View the Image

You can also simply open the image:

```
xdg-open vulture.jpg    # Linux
open vulture.jpg        # macOS
start vulture.jpg       # Windows
```

The flag may be visually visible or hidden in the metadata.

Step 13: Alternative – Using tshark (Command Line)

If you prefer the command line, you can decrypt and extract objects using `tshark`:

```
# Decrypt and extract HTTP objects
tshark -r capture.pcap -Y "http" -o "uat:rsa_keys:\"picopico.key\", \"\"" --export-objects "http,./exported"
```

Then search the exported files:


```
grep -r "picoCTF" ./exported/
```

Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, extracting `vulture.jpg` from the HTTP objects and running `strings` on it will reveal the complete flag.

Key Concepts

Concept	Description
TLS (Transport Layer Security)	Cryptographic protocol that encrypts HTTPS traffic.
RSA Private Key	Used to decrypt TLS sessions if the same key was used by the server.
Wireshark TLS Decryption	Feature that decrypts encrypted traffic when provided with the server’s private key.
HTTP Object Export	Wireshark feature to extract files transferred over HTTP.
Steganography in Images	Flags can be hidden as strings within image files.
Custom HTTP Headers	Some challenges put flags directly in headers (but not this one).

Pro Tips

- 1. **Don’t stop at the first flag candidate** – If the flag in the HTTP header is incorrect, look for transferred files.

2. **Export HTTP Objects** – This is often overlooked but can reveal downloaded files, images, scripts, and other artifacts.
3. **Run `strings` on extracted images** – Flags are often embedded as plain text within image files.
4. **Check the image visually** – Sometimes the flag is displayed in the image itself.
5. **Use `tshark --export-objects`** – This automates extraction and is scriptable.
6. **The key file works for decryption** – Even if the header flag is wrong, decryption still worked, allowing you to see all HTTP traffic.

Alternative Methods

Method	Command
Wireshark GUI	File → Export Objects → HTTP
tshark CLI	<pre>tshark -r capture.pcap -Y "http" -o "uat:rsa_keys:\"picopico.key\",\"\\" --export-objects "http,./exported"</pre>
Convert OpenSSH key to PEM	<pre>ssh-keygen -p -f picopico.key -m PEM</pre> (if Wireshark rejects it)

Comparison: WebNet0 vs WebNet1

Challenge	Flag Location
WebNet0	HTTP header <code>Pico-Flag</code>
WebNet1	Inside extracted image <code>vulture.jpg</code>

Both challenges require TLS decryption with the provided key. However, WebNet1 hides the flag deeper – you must export HTTP objects and examine the downloaded image.

Conclusion

The PCAP file `capture.pcap` contained encrypted TLS/HTTPS traffic. Using the provided RSA private key `picopico.key`, we configured Wireshark to decrypt the TLS stream.

After decryption, we first looked for the `Pico-Flag` HTTP header (as in WebNet0), but it was incorrect. Further investigation revealed an HTTP object named `vulture.jpg`. After exporting and saving this image, running `strings` on it revealed the actual flag.

This challenge demonstrates:

- **TLS decryption** – Using a private key to decrypt captured encrypted traffic
- **HTTP object extraction** – Recovering files transferred over HTTP
- **Multi-stage flag discovery** – Sometimes the flag isn't in the obvious place; you need to dig deeper
- **Image forensics** – Using `strings` to find embedded text in image files

Key takeaway: When a flag isn't where you expect it, look for transferred files. Export HTTP objects from Wireshark and examine each one with `strings`. Flags can be hidden in images, documents, or any other file transferred during the session.